



The PACE Field Guide

Parallel Action, Compounding Effort.
Two laws for getting real throughput out of
AI agents.

Wynter Jones

WYNTER.AI

The PACE Field Guide

01	The Bottleneck	3
02	Act in Parallel	6
03	Compound Effort	12
04	Why Both	17
05	The Toolkit	20
	The Dispatch Brief	21
	The Context File	23
	The PACE Protocol	24
	The Compounding Checklist	26
	A week in the life	27

HOW TO USE IT

Parts 1 to 4 are the argument. Part 5 is the part you act on — the dispatch brief, the context file, the drop-in protocol and the checklist. If you only have ten minutes, read page 6, page 12, and then go to page 21.

01

PART ONE

The Bottleneck

You were promised a multiplier and you got a faster typist. The reason is structural, and it has nothing to do with how you write prompts.

01

AI didn't 10x you. Here's why.

You have access to a system that writes code faster than you can read it. On paper you should be shipping four times what you shipped last year. You are shipping maybe 1.3 times. That gap is not a prompting problem.

Here is an honest hour, for most people using agents at work.

You write a short task. The agent starts. You watch it type. Three minutes in it asks a clarifying question you could have answered in the first message. You answer. You watch more. It finishes. You read the output, spot the thing it got wrong, correct it by hand. Then you open a new session and explain your project again from an empty context window.

Multiply that by a day. You did not get a multiplier. You got a faster typist and a new way to be interrupted.

Two things are broken there, and they are not the same thing.

FAILURE ONE

Your attention is single-threaded.

The agent can run concurrently. You cannot. You have one attention and you spent it watching a machine work. Every agent you are actively supervising runs at exactly the speed of one human — yours. The concurrency was there the whole time. You declined to use it.

FAILURE TWO

Your work doesn't accumulate.

Every session starts from zero. You re-explain the same architecture, restate the same conventions, re-correct the same misunderstanding about how your project handles errors. Task ten costs what task one cost, because nothing you learned in tasks one through nine got written down anywhere the agent can reach. That is not a workflow. It is a treadmill with good marketing.

Two failures, two laws.

PACE is not a productivity system. It is two constraints on how you set work up — one for each failure above.

1

Act in Parallel

FIXES THE ATTENTION PROBLEM

Stop being the thing your agents wait on. Structure work so it runs without you, and spend the time you get back setting up the next piece of work instead of watching this one.

2

Compound Effort

FIXES THE ACCUMULATION PROBLEM

Make every task leave something behind that makes the next task cheaper. Context written down, corrections folded back into the system, solved problems turned into things you can reuse. The residue is the point.

NOTE

Both laws cost you time at the front of a task and repay it later, which is why most people skip them. Ten minutes of specification does not feel like progress. It is the only thing on this list that reliably is.

And to be clear about what this isn't: none of it is about better prompts. Prompt wording matters roughly as much as handwriting mattered in 1990. Real, but not the thing holding you back.

02

PART TWO — LAW ONE

Act in Parallel

"Your agents can run concurrently. You can't. So stop being the thing they wait on."

PACE — LAW ONE

02

Plan once, at the front.

A vague task interrupts you every four minutes. A well-specified task runs for an hour.

That is the entire trade. Ten minutes of thinking before you dispatch buys an hour in which you are not needed. Skipping those ten minutes does not save ten minutes — it redistributes them across the next hour in four-minute fragments, which is the most expensive way there is to spend attention.

So front-load every decision the agent would otherwise have to stop and ask about:

- What “done” looks like, concretely enough to check.
- Which files and systems it may touch, and which it may not.
- What to do at an ambiguity. Not “ask me” — *assume, and flag it*.
- How it should verify its own work before it reports.

If you can't answer those four, the task isn't ready to dispatch. That is information, not a delay.

THE UNIT

The unit of work

Stop thinking in prompts. Start thinking in units, where the unit is: **a task an agent can finish without asking me anything.**

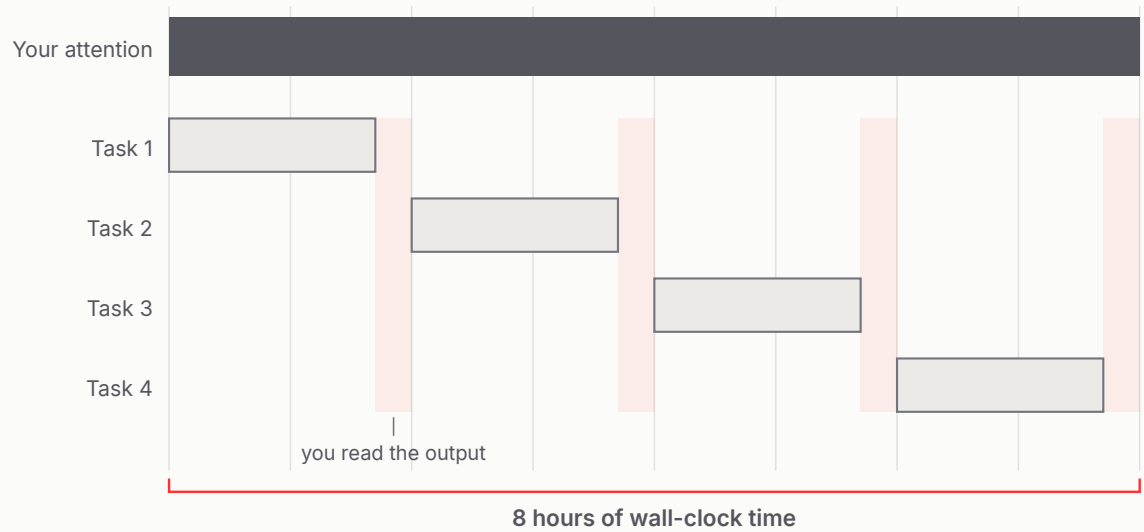
That definition does real work. It rules out “improve the onboarding flow” — too vague, it will be back with questions in four minutes. It rules out “rename this variable” — done in twelve seconds, not worth the dispatch overhead. What survives is the middle: substantial, bounded, unambiguous. Thirty to ninety minutes of agent time.

Most tasks fail the test on the first pass. Rewriting them until they pass *is* the work.

FIGURE 01 The same four tasks, twice

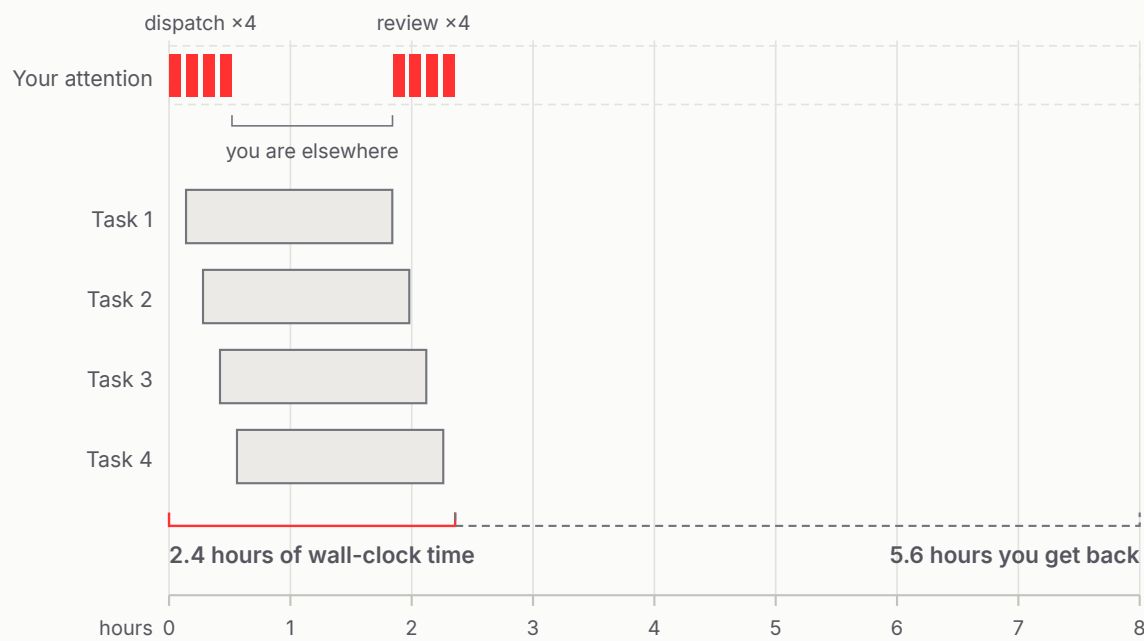
Serial — you stay in the loop

Four tasks, one attention, strictly one after another.



Parallel — you dispatch, then leave

The same four tasks, running at the same time.



Same four tasks, same total agent time, same person. The only variable is whether you stayed in the room while they ran.

Dispatch, then leave.

The moment you kick off agent A, your job is not to watch agent A. Your job is to go plan work for agent B.

This feels wrong, and the feeling is worth naming. Part of it is curiosity. Most of it is a suspicion that if you look away it will do something stupid. It might. Branches and checkpoints cover that.

What watching actually buys you is not safety. It is the sensation of control, and it costs you the entire parallel dividend.

Batch the planning. Parallelise the execution. Spend the first forty minutes of the morning writing four briefs. Dispatch all four. Spend the rest of the day reviewing what comes back and writing the next four. The desk is quiet in between, and that is the system working, not the system idle.

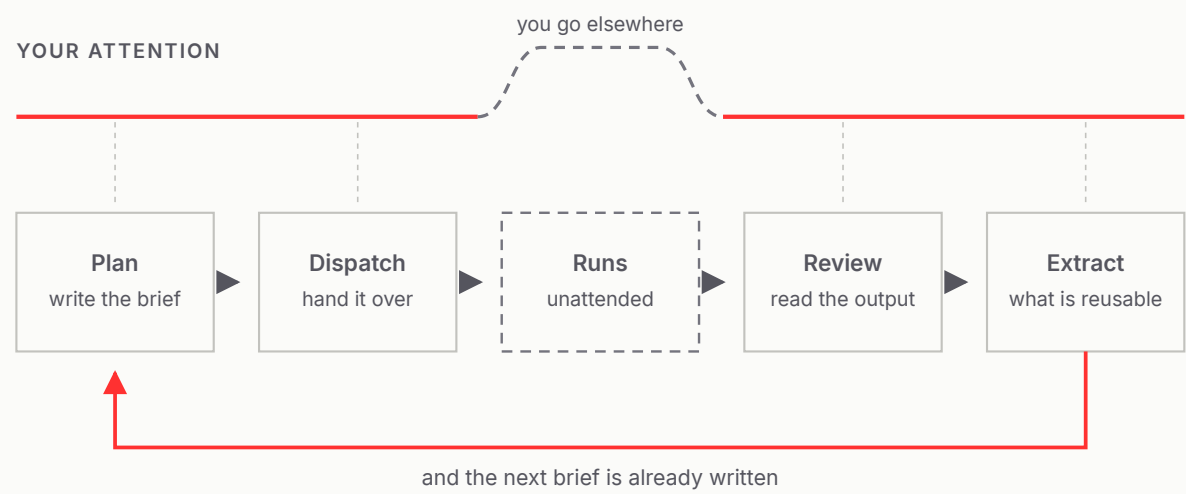
FAILURE DESIGN

Design for unattended failure

Running unattended means accepting that some runs will fail badly. The job is to make that cheap rather than to prevent it.

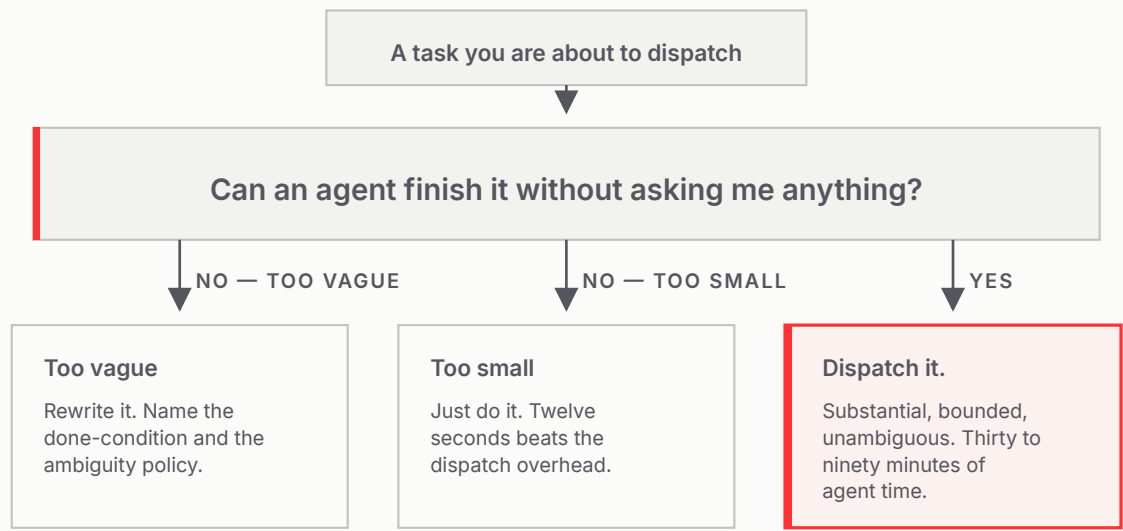
- **Checkpoints.** Every agent works on its own branch or worktree. A bad run is a discarded branch, not a lost afternoon.
- **Isolated workspaces.** Two agents editing the same file at the same time is not parallelism. It is a merge conflict with extra steps.
- **Scope written down.** One sentence — “do not touch the auth module” — prevents an entire category of disaster.
- **Self-verification.** If the agent can run the tests, it should have to run the tests before it reports.

FIGURE 02 The dispatch loop



The unattended stretch is the whole point. If your attention never leaves the line, you are having a conversation, not running a loop.

FIGURE 03 The unit-of-work test



Most tasks fail on the first pass, and both failures are useful information. Rewriting them until they pass is the work.

The skill is dispatch, not prompting.

Prompting is what you do when you are in the loop.
Dispatch is what you do when you are not.

Prompting optimises a conversation: how do I phrase this so the next reply is better? It assumes you will be there for the next reply. Dispatch optimises an absence: what does this agent need so that in ninety minutes I get a finished thing instead of a question?

They pull in different directions. A good prompt can afford to be short, because you are there to fill the gaps. A good dispatch brief cannot, because you aren't. Every hour spent learning to phrase things more cleverly is an hour spent getting better at being the bottleneck.

The move is not a better sentence. It is a task specified well enough that the conversation never has to happen.

METRIC — LAW ONE

Agent-hours per human-hour.

Count it honestly for one day: total hours of agent work performed, divided by hours you spent at the desk. Almost everyone lands at **1:1** — one agent, watched. The target is **4:1** and up.

1:1

One agent, supervised. A faster typist with a human attached.

4:1

Four unattended runs against one planning hour. Same effort, rearranged.

Going from 1:1 to 4:1 is not four times the work. It is the same work, front-loaded.

03

PART THREE — LAW TWO

Compound Effort

"If the tenth task isn't faster than the first, you don't have a system. You have a habit."

PACE — LAW TWO

03

Context is an asset.

Everything you know about your project that the agent doesn't is a tax you pay on every single task.

You pay it in re-explanation, in wrong assumptions, in corrections you have made before. Written down once, it stops being a tax and starts being infrastructure.

What belongs in writing: the architecture and why it is shaped that way. The conventions the code follows but never states. The commands to build, test and deploy. The things that look wrong and are deliberate. The decisions that are already closed, so they stop being reopened every fortnight.

Where it lives matters less than that it lives somewhere the agent reads — a context file at the project root, a runbook, a starter repo, the design system. Treat whichever one you use the way you would treat an onboarding document for a contractor who starts fresh every morning. That is precisely what you have.

ARTIFACTS

Every solved problem becomes a thing

You will solve some version of the same problem repeatedly for as long as the project exists. The compounding move is to make each solution retrievable instead of remembered.

- A brief that worked → a saved template.
- A component you finally got right → the reference implementation, named in the context file so the agent copies that one.
- A three-hour debugging session → four lines about the failure mode.
- A repeated multi-step chore → a documented, named procedure.

The bar is deliberately low. A named paragraph on disk beats an unnamed insight in your head, every time.

Correct the system, not the output.

This is the one that separates the people who compound from the people who are simply busy.

The agent produces something wrong. Say it reached for a date library you moved off six months ago. Your instinct is to fix that file. Thirty seconds, done, back to work.

You just bought a fix that expires. Tomorrow's agent has the same gap and makes the same mistake, and you pay the same thirty seconds again, forever, at a rate set by how often the subject comes up.

The compounding move costs sixty seconds instead of thirty. Fix the output, then open the context file and write the line that makes the mistake impossible: *"Dates: use X. Y was removed in March — do not reintroduce it."*

Every correction is a fork. One branch fixes today. The other retires the mistake. Take the second one consistently and your error rate falls without you working on it.

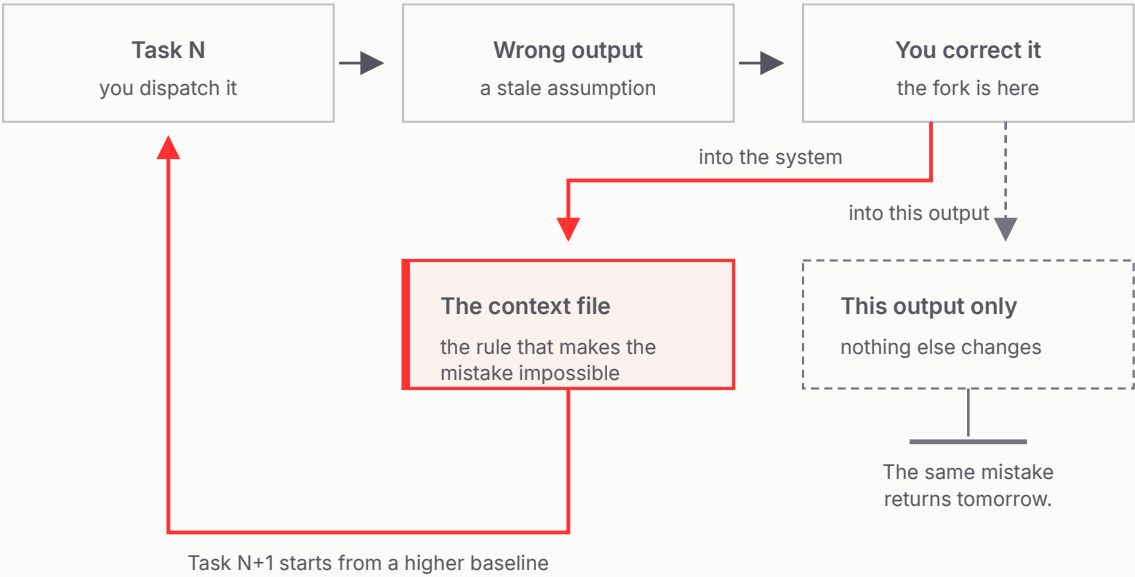
ZERO SETUP

Systems so setup is zero

If the start of every task involves decisions you have already made — spacing, component structure, file layout, naming — you are paying setup costs in perpetuity for no return.

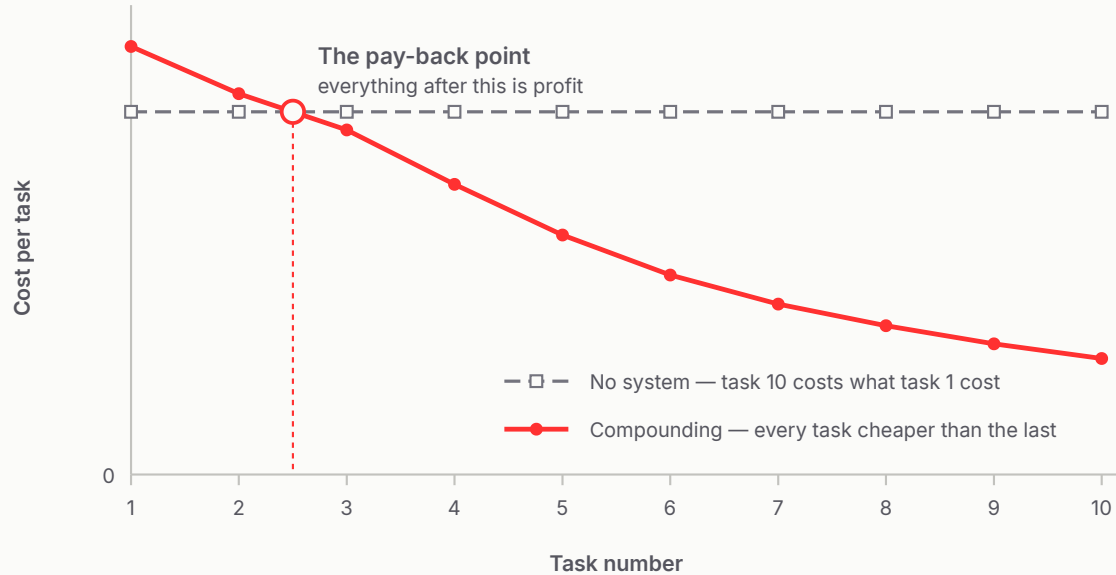
A design system, a project template, a scaffold command: these exist so that the answer to "how should this look" is already on disk and the agent can read it. Zero setup is a compounding position. Every task after it starts further along than the last one did.

FIGURE 04 Where a correction goes



Every correction is a fork. One branch fixes today; the other retires the mistake. They cost thirty seconds and sixty.

FIGURE 05 What compounding looks like



The up-front cost is real: task one is more expensive under a system than without one. It is paid back somewhere around task three.

Audit and prune.

Stale context is worse than no context.

No context makes the agent ask. Stale context makes it confidently wrong — and you will not catch it, because the output looks like it followed instructions. It did. They were the wrong instructions.

Everything you have written down rots the way comments rot — the context file, the runbooks, the template you saved in March. Once a month, or after any significant refactor, read them end to end and delete everything that is no longer true. Deleting is the job. A document that only ever grows becomes one nobody trusts to be current, which is operationally the same as not having one.

Delegate the audit. *“Read this context file against the current code and tell me what’s stale”* is a five-minute task with a high hit rate.

METRIC — LAW TWO

Is task N faster than task 1?

Pick a kind of work that recurs — a new endpoint, a new screen, a new report — and write down what the first one cost you. Time it again on the tenth. If the number hasn’t moved, nothing you did in between accumulated. You have a habit, not a system.

Task 1

Full cost. Every decision made from scratch.

Task 10

Should be a fraction. If it isn’t, find what you failed to write down.

04

PART FOUR

Why Both

Each law fails on its own, and it fails in a way you will recognise. Together they run as a loop.

04

One law is a half-measure.

Parallel without compounding is chaos.

Five agents running at once, every one of them missing the same context, every one making the same three mistakes, all of them reporting within twenty minutes of each other.

You multiplied your output and you multiplied your correction load with it. The correction load wins. This is the setup that looks extremely impressive in week one and has you burnt out by week five — and the failure gets misread as “agents don’t work on real projects.”

Compounding without parallel is one slow human.

Immaculate context files. Templates for everything. Every problem you have ever solved, documented and findable.

And one agent at a time, watched from start to finish, because the operator never stopped being the bottleneck. Genuinely better than where you started — and permanently capped at the speed of one person’s attention.

TOGETHER

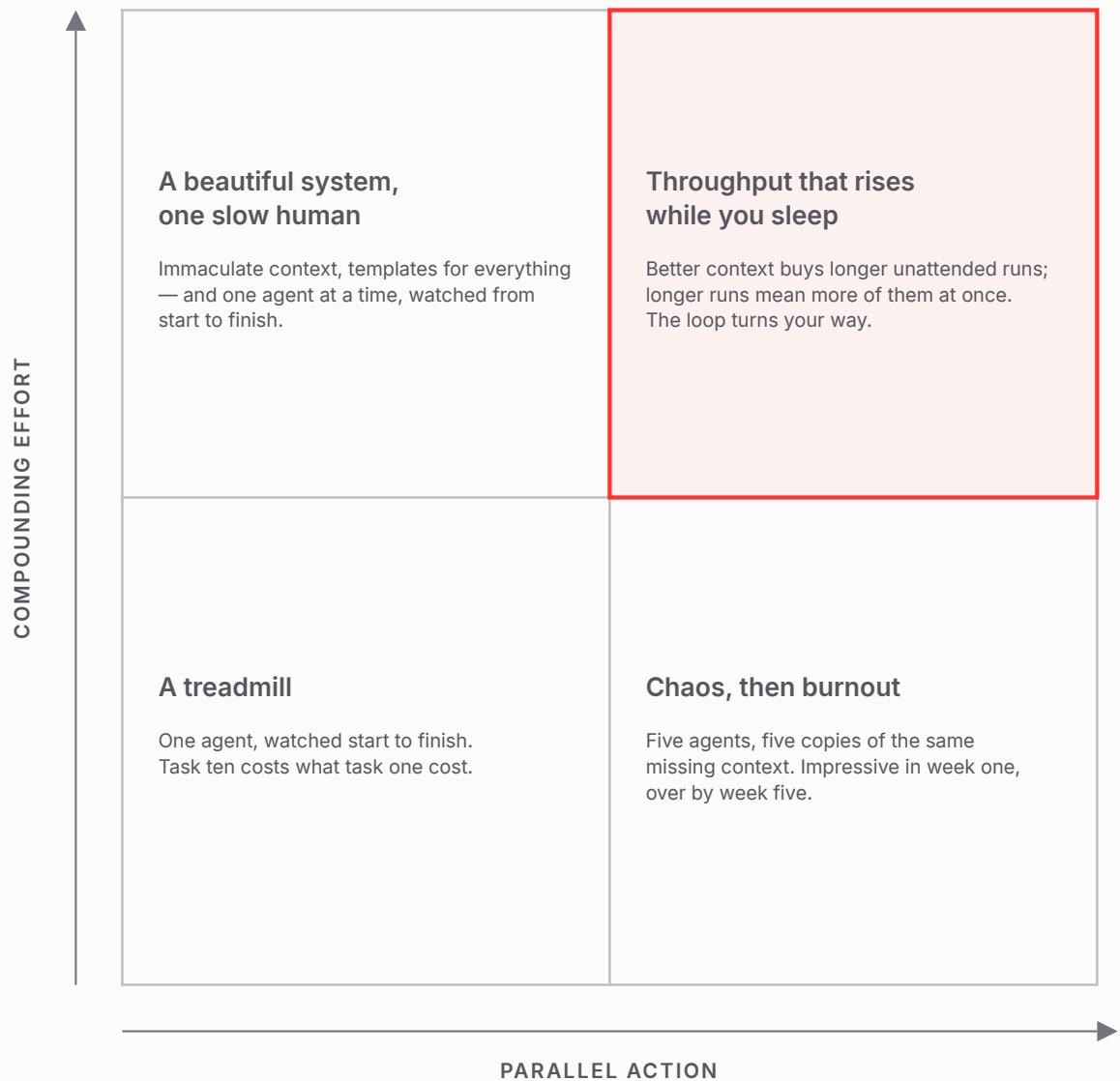
The loop

Compounding raises the quality of what every parallel agent starts with. Better starting context means each agent can run longer before it needs you. Longer unattended runs mean you can keep more of them going at once. More agents running means more problems solved, which means more artifacts, which means better context.

It is a loop, and it turns in the direction you want. That is the whole reason the two laws ship together rather than as two tips.

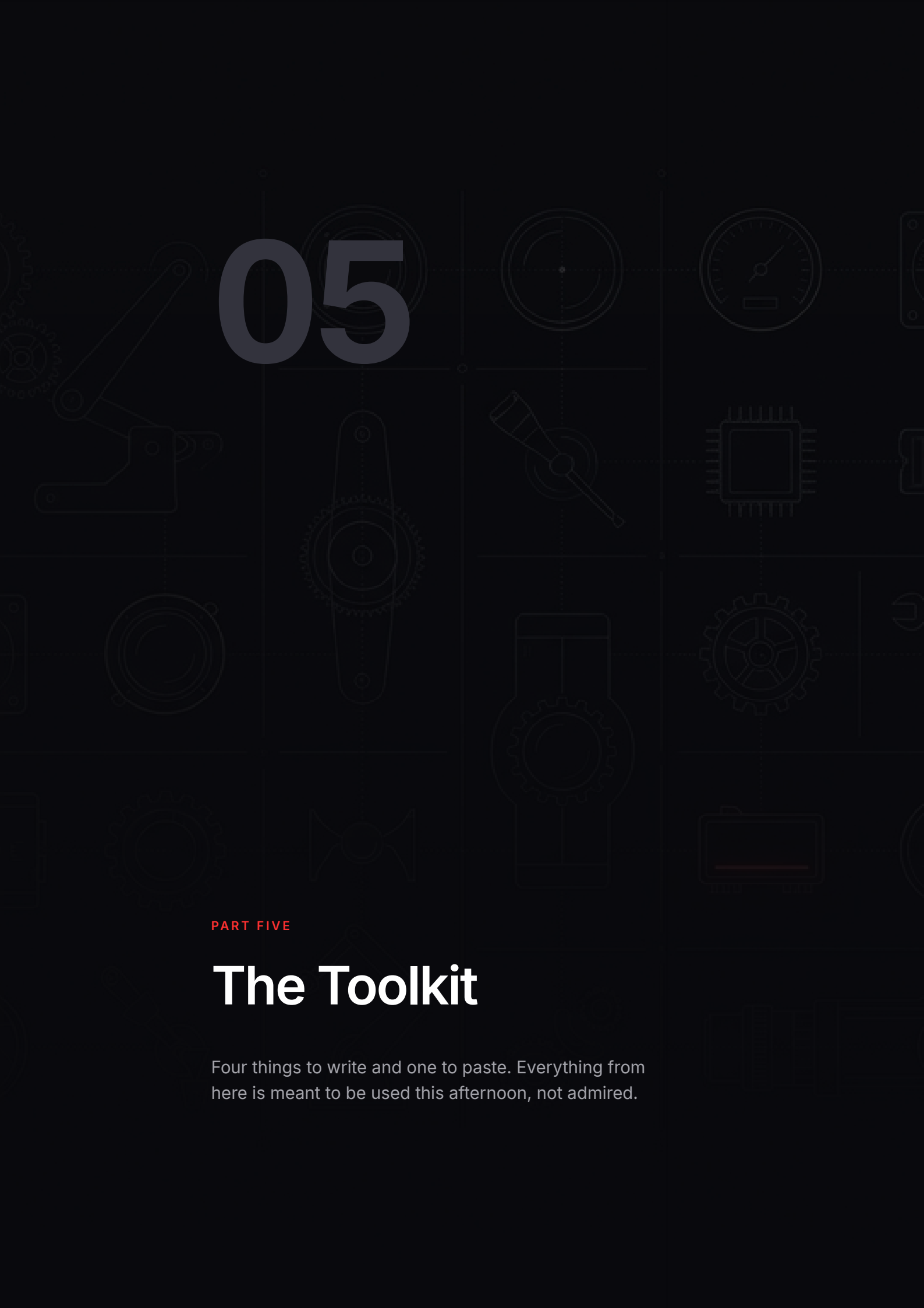
Throughput that increases while you sleep is not a slogan. It is just what a system looks like once the same corrections stop repeating.

FIGURE 06 Why it has to be both



Each law on its own moves you one square. Neither of them, on its own, gets you to the top right.

05

The background of the entire page is a dark charcoal grey. Overlaid on this is a light grey grid. Within the grid cells, there are faint, white line-art icons of various mechanical and technical components. These include gears of different sizes, a robotic arm, a pressure gauge, a microchip, a piston, a valve, a camera, a book, a magnifying glass, and various pulleys and belts. The icons are arranged in a somewhat random but systematic fashion across the grid.

PART FIVE

The Toolkit

Four things to write and one to paste. Everything from here is meant to be used this afternoon, not admired.

05

The Dispatch Brief

A task written so an agent can work on it for an hour without asking you a question. Longer than a prompt. Much shorter than a spec.

Six sections, in this order. The order matters: the agent reads top to bottom and should know what it is building before it learns what it cannot touch.

- 1 **Objective.** What exists at the end, in one sentence. If you need two sentences, you may have two tasks.
- 2 **Context.** Where things live, what is already true, and what it must read before starting. Pointing at an existing implementation and saying “match this” is the cheapest way to transmit fifty conventions at once.
- 3 **Constraints.** What it may not touch, change, or install. The most-skipped section, and the one that prevents the worst outcomes.
- 4 **Definition of done.** Checkable by a machine wherever possible. “Tests pass and the page renders” beats “works well.”
- 5 **Ambiguity policy.** One line, and it is always roughly the same line: ask everything blocking in one message now, then proceed on stated assumptions and flag them at the end.
- 6 **Report format.** What you want back, so reviewing it takes two minutes instead of twenty.

OVERLEAF

A real one, annotated. It took four minutes to write and it bought an hour of work nobody had to supervise.

An annotated brief

Left: the brief exactly as it was sent. Right: what each section is doing, and which interruption it removes.

OBJECTIVE

Add a CSV export to the Reports page: a button that downloads the currently filtered table as `reports-YYYY-MM-DD.csv`.

Names the artifact and the filename format. Two details that would otherwise each cost a round trip.

CONTEXT

Table is in `app/views/reports/index.html.erb`; filtering happens in `ReportsController#index`. There is an existing export on the Invoices page (`InvoicesController#export`) — match its approach, including how it names files and streams the response.

“Match this” carries every convention in that file without listing one of them.

CONSTRAINTS

Server-side only, no new gems. Don’t change the filtering logic. Don’t touch `InvoicesController`.

“No new gems” is three words that prevent an hour of dependency archaeology.

DONE WHEN

Export respects every active filter. The file opens in Excel without a mangled first column. A controller test covers the filtered case. `bin/rails test` is green.

The Excel detail is a specific failure the author has been bitten by. Specific beats thorough. A green suite is machine-checkable, so the agent can verify itself.

AMBIGUITY

Columns: mirror the visible table columns. Anything else blocking, ask it in one message now; otherwise assume, proceed, and flag at the end.

Pre-answers the single most likely blocking question, then licenses the agent to keep moving past the rest.

REPORT

What changed, what you assumed, anything left open.

Three headings. Review takes two minutes.

Reusable, too: next quarter’s export brief is this one with three words changed.

The Context File

What the project knows has to sit where the agent will find it. Start with one file, at the root, read first in every session. Name it for whatever tool you use.

CLAUDE.md

AGENTS.md

.cursorrules

.windsurfrules

BELONGS**What goes in**

- What this project is, in three sentences.
- The stack, and the version numbers that actually constrain decisions.
- Commands: build, test, run, deploy.
- Conventions the code follows and never states.
- Deliberate oddities — *this looks wrong, here is why it stays*.
- Closed decisions, so they stop being reopened.
- Where to look first for the kinds of change that come up most.

DOESN'T**What stays out**

- Anything the code states more accurately than prose can. Don't restate the schema — point at it.
- Aspirations. Describe the codebase you have, not the one you meant to build.
- History, unless it is load-bearing as *don't reintroduce X*.
- Volume. A file nobody trusts to be current gets skipped — by the agent, and by you.

UPKEEP

Append whenever you correct something this file should have prevented. Prune monthly. Keep it under two pages; past that, split it and leave pointers.

The PACE Protocol

The drop-in block. Paste it verbatim into your context file and both laws become the default in every session.

PASTE VERBATIM

CLAUDE.md / AGENTS.md / .cursorrules / .windsurfrules

PACE – How We Work Here

This project runs on PACE: ****Parallel Action, Compounding Effort.****
Apply both laws in every session.

Law 1 – Act in Parallel

My attention is the bottleneck, not your speed. Work so I don't have to watch you.

- Front-load the questions. Before you start, restate the task, state your assumptions, and ask everything blocking in ONE message. After that, proceed on your stated assumptions rather than stopping to ask – flag them at the end.
- Plan first, then run. State the plan, then execute it end to end. Report once when you're done, not after every step.
- Aim to run unattended. If a task is too vague to work on for a long stretch without checking in, say so and propose how to split it into pieces that can.
- Never stall. If one piece is blocked, park it with a note, move to the next independent piece, and raise the block in your final report.
- Don't ask "what's next" – check the plan or backlog and take the next unblocked item.
- Verify your own work before reporting. Run it, test it, read it back.

INSTALLING IT

Three things worth knowing

- Put it at the **top** of the context file, above your project-specific notes. It sets the working posture; everything below it is detail.
- It does not replace your own context. It governs *how* the agent works; your file still has to say what the project is.
- Paste it as one block, including Law 2 overleaf. The two laws are load-bearing on each other and half of it is worth much less than all of it.

The PACE Protocol

Continued.

PASTE VERBATIM — CONTINUED

wynter.ai/pace/protocol

Law 2 — Compound Effort

The tenth task should be faster than the first. Leave the system better than you found it.

- Read the existing context first — this file, the README, the runbooks, the code's own conventions. Match what's there. Don't introduce a second way of doing something that already has a way.
- Correct the system, not the output. If you got something wrong because the context was missing or stale, fix the context file too, not just this answer.
- Extract what's reusable. When you solve something that will come up again, make it a template, a component, a helper, or a documented pattern — and say what you extracted.
- Prefer existing abstractions and the established design system. A new pattern needs a stated reason.
- Flag rot. If something in this file is stale or contradicts the code, say so.

Every substantial task ends with

1. What changed
2. What you assumed
3. What's still open or blocked
4. What you'd add to the context files so the next run is faster

PACE — Parallel Action, Compounding Effort. Free framework: <https://wynter.ai/pace>

COPY IT

The same block, in a form you can select and copy in one click:
wynter.ai/pace/protocol

The Compounding Checklist

What should exist in your system and probably doesn't yet. Tick honestly.

- ☐ A context file at the project root, read first in every session.
- ☐ The PACE protocol pasted into it.
- ☐ At least one dispatch brief saved as a reusable template.
- ☐ A named reference implementation for your most common kind of change.
- ☐ Build, test and deploy commands written where the agent will read them.
- ☐ A design system or scaffold, so a new screen starts at zero setup.
- ☐ Branch or worktree isolation, so a bad unattended run costs nothing.
- ☐ A verification step the agent runs before it reports.
- ☐ A standing report format: changed, assumed, open, context to add.
- ☐ A recurring calendar entry to prune the context file.
- ☐ One number you actually track: agent-hours per human-hour.
- ☐ One recurring task you have timed twice, to prove task N beats task 1.

SCORING

Counted honestly, most people have two or three. That is not a criticism — every one of these is invisible until it exists, and none of them are urgent on any given Tuesday.

Every box you tick pays out on every task after it. Start with the first two. They take twenty minutes together and they change the shape of every session that follows.

A week in the life

Both laws running at once. Deliberately unglamorous — the compounding is invisible on any single day.

MONDAY

08:40. Not writing code. Writing four briefs — a CSV export, a flaky-test triage, a settings page, a documentation pass. Ninety minutes. Feels unproductive; it is the most leveraged block of the week.

10:15. All four dispatched, each on its own branch. The desk goes quiet. Writing the settings brief surfaced a decision nobody had actually made, so it gets made — and it goes in the context file, not just in the brief.

TUESDAY

Three of four came back clean. The docs pass invented a section that doesn't exist, because the context file still describes a module removed in April. Fix the docs, then delete the module from the context file. The second part is the part that counts.

Afternoon: four new briefs. The settings brief is now a template, so the next screen brief takes eight minutes instead of thirty.

WEDNESDAY

The flaky test was a real race condition, not flakiness. Fixed. Four lines about the failure mode go into the context file, because it will happen again somewhere else.

THURSDAY

Six agents, not four. Nothing got harder. The briefs are faster to write because half of each one is now a pointer to something already written down.

FRIDAY

16:00. Counting: roughly twenty-six agent-hours against six hours at the desk. Two months ago the same week ran at one to one, and it did not feel any easier.

Nothing there is impressive on a single day. That is the point. Compounding is invisible in a week and obvious in a quarter.

That's the whole thing.

Two laws. Stop being the thing your agents wait on. Make sure the tenth task is faster than the first.

Everything in this guide is downstream of those two sentences. The briefs, the context file, the checklist, the protocol — they are all just ways of making the two laws survive contact with a normal working week, when you are tired and it would be easier to sit and watch the agent type.

You can start this afternoon, and it does not require a new tool. Write one brief good enough that you can genuinely walk away from it. Then write down one thing you have already explained twice.

That is a 2:1 day and the first line of a context file. Everything else compounds from there.

A handwritten signature in black ink that reads "Wynter Jones". The signature is fluid and cursive, with a long horizontal stroke extending from the end of the name.

Wynter Jones

wynter.ai



**The protocol, free, in a form you
can copy in one click.**

wynter.ai/pace/protocol

Paste it into your context file and both laws become the default in every session you run after that.